

Data Quality: Some Comments on the NASA Software Defect Datasets

Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carolyn Mair

Abstract—Background—Self-evidently empirical analyses rely upon the quality of their data. Likewise, replications rely upon accurate reporting and using the *same* rather than *similar* versions of datasets. In recent years, there has been much interest in using machine learners to classify software modules into defect-prone and not defect-prone categories. The publicly available NASA datasets have been extensively used as part of this research. Objective—This short note investigates the extent to which published analyses based on the NASA defect datasets are meaningful and comparable. Method—We analyze the five studies published in the *IEEE Transactions on Software Engineering* since 2007 that have utilized these datasets and compare the two versions of the datasets currently in use. Results—We find important differences between the two versions of the datasets, implausible values in one dataset and generally insufficient detail documented on dataset preprocessing. Conclusions—It is recommended that researchers 1) indicate the provenance of the datasets they use, 2) report any preprocessing in sufficient detail to enable meaningful replication, and 3) invest effort in understanding the data prior to applying machine learners.

Index Terms—Empirical software engineering, data quality, machine learning, defect prediction

1 INTRODUCTION

PRESENTLY, there is a good deal of interest in using machine learning methods to induce prediction systems to classify software modules as faulty or not faulty. Accurate prediction is useful because it enables, among other things, testing resources to be targeted more accurately. A 2009 Mapping Study [4] identified 74 relevant studies, and this had grown to 208 by the end of 2010 (as reported by Hall et al. [7]). These studies have employed a range of methods (e.g., Bayesian, support vector machines, and instance-based learners) and applied them to different software defect datasets.

With so much research being undertaken, there is a clear need to combine individual results into a coherent body of knowledge. To accomplish this, it is necessary to make valid comparisons between studies. This is facilitated where studies have used the same datasets. It also assumes that results are derived from meaningful data. However, a recent paper by Gray et al. [6] has raised important questions about the quality of the 13¹ software defect datasets that have been made publicly available and extensively used by

researchers (e.g., Hall et al. [7] found more than a quarter of relevant defect prediction studies, that is, 58 out of 208, made use of the NASA datasets). Therefore, these concerns about data integrity and inconsistencies between different versions of the NASA datasets in circulation require urgent attention. To do otherwise undermines the scientific basis of empirical validation and replication of studies of software defect prediction.

This note builds upon the initial work of Gray et al., who pointed out the quality problems with the NASA datasets currently in use by the research community. We consider the extent of the problem, describe in detail a preprocessing algorithm and the impact of various cleaning ordering issues, make different cleaned datasets publicly available, and conclude with suggestions as to how the research community might avoid such problems in the future.

2 INVESTIGATION AND RESULTS

Machine learning is a data-driven form of research and so it comes as no surprise that datasets are archived and shared between researchers. In this regard, the Promise Data Repository² has served an important role in making software engineering datasets publicly available. For example, presently (14 June 2012), there are 96 software defect datasets available. Among these are 13 out of the 14 datasets that have been provided by NASA, which were also available for download from the NASA Metrics Data Program (MDP) website.³

Table 1 compares the two versions and in terms of cases (instances) and features (attributes). All data are for the raw, i.e., unprocessed, versions of the files. We see that no two versions are identical, although the scale of the differences varies considerably. We also note that the ordering of cases

1. There are presently 14 datasets, but note that KC2 was not present on the MDP website and KC4 is not present in the Promise Data Repository.

- M. Shepperd is with the Department of IS and Computing, Brunel University, Uxbridge, UB8 3PH, United Kingdom. E-mail: martin.shepperd@brunel.ac.uk.
- Q. Song and Z. Sun are with the Department of Computer Science, Xi'an Jiaotong University, PR China. E-mail: qbsong@mail.xjtu.edu.cn, zhongbin@stu.xjtu.edu.cn.
- C. Mair is with the Graduate School, London College of Fashion, The University of the Arts, 20 John Prince's Street, London W1G 0BJ, United Kingdom.

Manuscript received 9 Nov. 2011; revised 28 June 2012; accepted 12 Nov. 2012; published online 15 Feb. 2013.

Recommended for acceptance by R. Jeffery.

For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number TSE-2011-11-0317. Digital Object Identifier no. 10.1109/TSE.2013.11.

2. See <http://promisedata.org>.

3. Presently, they may be found at <http://www.filesanywhere.com/fs/v.aspx?v=896a648c5e5e6f799b>.

TABLE 1
Comparison of the Two Versions of the NASA Defect Datasets

Data Set	Cases		Features	
	MDP	Promise	MDP	Promise
CM1	505	498	43	22
JM1	10878	10885	24	22
KC1	2107	2109	27	22
KC2	n.a.	522	n.a.	22
KC3	458	458	43	40
KC4	125	n.a	43	n.a
MC1	9466	9466	42	39
MC2	161	161	43	40
MW1	403	403	43	38
PC1	1107	1109	43	22
PC2	5589	5589	43	37
PC3	1563	1563	43	38
PC4	1458	1458	43	38
PC5	17186	17186	42	39

within the datasets differs. This may impact validation strategies such as n -fold cross validation if random folds are not utilized.

Next, we consider the different types of data quality problem that might arise. These are defined in Table 2 and employed in Tables 3 and 4, which analyze the data quality issues of each dataset from the NASA MDP repository and Promise repository in more detail. Column A refers to a situation where two or more features contain identical values for all observations, i.e., for all cases. Column B refers to features that contain the same value, i.e., add no information. Column C counts the number of features that contain one or more missing observation, and Column I counts the number of instances with missing values. We provide both values since both list and casewise deletion are possible remedial strategies. For the same reason, we give both feature (Column D) and case (Column J) counts of conflicting values. These arise when some implied relational integrity constraint is violated, for example, LOC_TOTAL cannot be less than commented LOC since the former must subsume the latter. Another instance is where the McCabe's $v(G)$ is 128 for a module yet the count of executable lines of code is zero (see [5] for a brief tutorial article on integrity constraints). Next, Columns E and K give the counts of the number of features and cases containing one or more implausible values such as LOC = 1.1. The checks are described in the Appendix. The data quality problems are then summarized by the total number of features impacted (Column F) and cases (Column L).

Finally, note that all datasets “suffer” from problems of duplicate cases. Some researchers have considered this to be a problem since the identical case may be used both for training and validation (e.g., [6], [10]); however, we do not fully concur because our view is that it depends upon the

goal of the investigation. If one is concerned with generalization to other settings and datasets, then elimination of duplicate cases has some basis since duplicate instances may not be commonplace and will tend to lead to an overoptimistic view of predictive capability. If the research goal is to form a view of how the classifier will perform in a particular setting, then naturally occurring duplicate cases (i.e., different software modules indicated by different module ids with similar profiles) offer a learning opportunity (since a previously encountered, identical case should facilitate learning). We would also argue likewise with inconsistent or conflicting cases. This leads to challenges for any learner. Removing them may distort results to being overoptimistic. The researchers' choices again depend upon the investigation goal.

Table 5 lists five empirical studies that have been published in the *IEEE Transactions on Software Engineering (TSE)* since 2007, of which three out of five report using the MDP versions of the datasets. This was established by e-mailing the authors of each study. Of course, many other studies have been published elsewhere, but for brevity we focus on the community's flagship journal. What is clear is that there are differences in the base level dataset version and in reporting detail. Unfortunately, this hinders making sense of the combined results and building an overall body of knowledge. Unless these problems are resolved, any attempt at meta-analysis will be compromised.

3 CLEANED VERSIONS OF THE NASA DATASETS

In this section, we address the problems identified previously, describe the preprocessing involved, and make new versions of the datasets available for other researchers. This will enable a common basis for research and meaningful comparison between studies.

The preprocessing strategy is that first the problem data (e.g., cases with either conflicting feature values or implausible values) are discarded, and then the data, which are not problematic but do not help improve the defect prediction (e.g., the features with constant values and either identical or inconsistent cases), are removed. This results in data set DS being transformed to DS' and DS'' , respectively. Procedure NASA MDP data preprocessing approach provides the details in Fig. 1.

The preprocessing algorithm consists of two parts: The first part (lines 3-24) deals with cases, while the second part (lines 25-31) handles features. Cases with either implausible values or conflicting feature values are logically erroneous: They either are implausible or contain features that violate some referential integrity constraint, so they are removed first (lines 3-10). The identical cases may constitute problems as a consequence of the cross-validation strategy. The inconsistent cases are problematic because it is not obvious how the learner should be trained. Thus, they are also both deleted (lines 11-20). Note that the preprocessing order of these two situations cannot be swapped; otherwise, some inconsistent cases may not be removed. For example, suppose cases i and j are a pair of inconsistent cases, and case k is identical to case i ($i < j < k$). Thus, cases k and j are also a pair of inconsistent cases, and all three of these cases should be removed. However, if cases i and j are removed first, then case k might not be removed as there is

TABLE 2
Data Quality

Column Label	Data Quality Category	Explanation	Example
A	Identical features	Refers to a situation where two or more features contain identical values for all cases.	$F1=F2=F3 \wedge F4=F5 \implies$ 3 features are identical so could be deleted.
B	Constant features	Refers to features that contain the same value for every instance, i.e. add no information.	
C	Features with missing values	Counts the number of features that contain one or more missing observations.	$F1$ has 10 missing values $\wedge$ $F3$ has 3 missing values $\implies$ 2 features contain missing values.
D	Features with conflicting values	Counts features that violate some referential integrity constraint	$F1$ should equal $F2+F3$ but does not. We cannot say which feature is in error therefore $\implies$ 3 problematic features.
E	Features with implausible values	Counts features that violate some integrity constraint	$F1$ should be non-negative but contains 1 or more instances $< 0 \implies$ 1 problematic feature.
F	Total problem features	Count of features impacted by 1 or more of A-E. Since features may contain more than one problem this need not be the sum of A to E .	
G	Identical cases	Refers to a situation where two or more cases contain identical values for all features including class label.	
H	Inconsistent cases	As per G but the class labels differ, all other data item values are identical.	There are two identical modules $M1$ and $M2$ where $M1$ is labelled as fault-free and $M2$ is labelled as faulty.
I	Cases with missing values	Counts the number of cases that contain one or more missing observations	
J	Cases with conflicting feature values	Counts cases that contain features (2 or more by definition) that violate some referential integrity constraint. Count each case irrespective of the number of features implicated.	As per Column D
K	Cases with implausible values	Counts cases that violate some integrity constraint. Count each case irrespective of the number of features implicated.	As per Column E.
L	Total of data quality problem cases	Count of cases impacted by one or more of I to K that we denote DS' . Since cases may contain more than one problem this need not be the sum of I to K.	
M	Total problem cases according to [6]	Count of cases impacted by one or more of G to K denoted DS''	

no longer any case that is inconsistent with case k . Lines 21-24 delete the cases with missing values. Finally, features with constant and identical values are removed (lines 25-31).

By applying the cleaning algorithm *NASA MDP data preprocessing approach* to the original NASA datasets, we obtain the corresponding new version of the datasets, which

were until recently available from their website. Note that Gray et al. [6] suggest an alternative cleaning algorithm; however, as we have observed, there are ordering effects, and we think it is better to remove demonstrably incorrect data items first as this may resolve conflicting feature values and so less data are lost in the cleaning process. Such

TABLE 3
Detailed Data Quality Analysis of the NASA Defect Datasets by Features

	A		B		C		D		E		F	
	Identical features		Constant features		Features with missing values		Features with conflicting values		Features with implausible values		Total problem features	
Data Set	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom
CM1	2	0	3	0	1	0	2	14	0	6	6	15
JM1	0	0	0	0	0	5	9	15	0	6	9	16
KC1	0	0	0	0	0	0	4	15	0	6	4	16
KC2	n.a.	0	n.a.	0	n.a.	0	n.a.	14	n.a.	6	n.a.	15
KC3	0	0	1	0	1	0	0	0	1	1	3	1
KC4	27	n.a.	26	n.a.	0	n.a.	3	n.a.	0	n.a.	30	n.a.
MC1	0	0	1	0	0	0	3	3	1	1	5	4
MC2	0	0	1	0	1	0	0	0	0	0	2	0
MW1	2	0	3	0	1	0	0	0	0	0	4	0
PC1	2	0	3	0	1	0	4	14	1	6	8	15
PC2	3	0	4	0	1	0	2	2	1	1	8	3
PC3	2	0	3	0	1	0	2	2	1	1	7	3
PC4	2	0	3	0	0	0	7	7	1	1	11	8
PC5	0	0	1	0	0	0	3	3	1	1	5	4

TABLE 4
Detailed Data Quality Analysis of the NASA Defect Datasets by Case

	G		H		I		J		K		L		M	
	Identical cases		Inconsistent cases		Cases with missing values		Cases with conflicting feature values		Cases with implausible values		Total problem cases, DS'		Total problem cases, DS''	
Dataset	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom	MDP	Prom
CM1	26	94	0	2	161	0	2	3	0	1	161	3	178	61
JM1	2628	2628	889	889	0	5	1287	1294	0	1	1287	1294	3158	3165
KC1	1070	1070	253	253	0	0	12	14	0	1	12	14	945	947
KC2	n.a.	182	n.a.	118	n.a.	0	n.a.	38	n.a.	1	n.a.	38	n.a.	197
KC3	12	170	0	2	258	0	0	0	29	29	258	29	264	142
KC4	10	n.a.	9	n.a.	0	n.a.	125	n.a.	0	n.a.	125	n.a.	125	n.a.
MC1	7972	7972	106	106	0	0	189	189	4841	4841	4841	4841	7619	7619
MC2	4	6	0	2	34	0	0	0	0	0	34	0	36	5
MW1	15	36	5	7	139	0	0	0	0	0	139	0	152	27
PC1	85	240	13	13	348	0	3	26	48	49	355	74	411	196
PC2	984	4621	0	100	4004	0	129	129	1084	1084	4055	1163	4855	4297
PC3	79	189	6	9	438	0	2	2	52	52	444	54	490	138
PC4	166	166	3	3	0	0	60	60	111	111	112	112	182	182
PC5	15730	15730	1725	1725	0	0	185	185	1772	1772	1782	1782	15507	15507

TABLE 5
Recent Software Defect Studies Published in *TSE* Based on the NASA Datasets

Study	Year	Version	Pre-processing		
			Missing items	Inconsistent items	Duplicate cases
Menzies, Greenwald and Frank [16]	2007	Promise	×	×	×
Zhang and Zhang [21]	2007	n.a.	n.a.	n.a.	n.a.
Lessman, Baesens, Mues and Pietsch [12]	2008	MDP	×	×	×
Liu, Khoshgoftaar and Seliya [15]	2010	MDP	✓	✓	×
Song, Jia, Shepperd, Ying and Liu [19]	2011	MDP	✓	×	×

considerations illustrate the subtleties of data cleaning. The cleaned datasets are available from <http://j.mp/scvvIU>.

4 DISCUSSION

Our analysis of datasets used for defect prediction research poses two questions. Do the data quality and differences between datasets matter in any practical sense? And if so, what should researchers do about it?

This paper raises some difficulties concerning the extensively used, collection of publicly available NASA software defect datasets. However, we consider it raises some significant and more widespread difficulties about how we conduct research into software defect prediction. This is for three reasons:

1. There is a growing movement toward ensuring that computational science (including machine learning) should generate reproducible results [20], [8], which is an undeniably good thing. A mainstay for this is sharing data and code. Such initiatives are undermined when differences, even subtle ones, between versions of the “same” data are used, either due to different preprocessing or to version control issues. When these differences are undetected by the research community, the problems deepen.
2. The differences between versions of some of the NASA datasets are not large. Nevertheless, it adds to the variance of the results so minimally it will make it more difficult to observe patterns across experimental results and will confound meta-analyses since a reduced proportion of the variability of the response variable (accuracy, however measured, of the predictors under investigation) due to the treatments (different choices of learning algorithm, dataset, and so forth).
3. Generally, we are dealing with small effects [17] but large samples (typically, in the thousands or tens of thousands); consequently, even small differences in training and validation data can lead to statistically significant differences in results. This in itself may be a reason to pay more attention to effect sizes and less to null hypothesis testing and p values [1].

Given that we believe data quality problems can matter considerably, we now move to the question of what might be done about it.

First, while sharing and making data available to members of the research community is clearly a good thing, given the possibility of differences being injected through copying and sharing, researchers should indicate the source of their data. As confirmation that this is not an isolated problem, consider the proliferation of *differing* versions of Fisher’s famous Iris dataset that has been used to explore linearly inseparable classification since the 1920s [2].

Second, as the methods of computational science become increasingly involved and demanding, great care is needed to ensure sufficient attention is paid to the data *as well as* the algorithms. Many machine and statistical learning methods are intricate and require a good deal of skill. This can divert attention away from data issues. In addition, there is a danger that secondary data analysis distances the researcher from the real-world phenomenon represented by the data. The meaning of the data can be lost and researchers may not know or ask what is meaningful. Consider the situation of a data item of zero LOC. Is this plausible? In some programming languages and depending upon how LOC is defined in the first place, it is possible though unlikely. Being distanced from the data collection makes it hard to answer these kinds of questions. We are not arguing against secondary analysis, but pointing out that it does bring some attendant dangers. Consequently, detailed documentation of the data is essential.

For example, we have observed, in common with Boetticher [3], some of the datasets contain implausible values such as $LOC = 1.1$. Given that some of these have occurred in the first case and feature (e.g., datasets CM1 and PC1), it is striking that this has elicited so little comment from those using these files. We collectively must be more zealous at policing the quality of the data that drives our research. As Jian et al. [9] state, “as we present our research results, we rely on the integrity of metric collection process and the description of software metrics reported in MDP repository.” The problem is compounded with duplicate versions of datasets that turn out to be inexact duplicates.

Therefore, the role of groups such as the Promise Data Repository, which manages public archives of datasets, needs to be extended to embrace data quality issues. Systematic reviews on how data quality is handled within empirical software engineering by Liebchen et al. [14], [13], [18] indicate that, presently, there is diversity in approach and scope for improvement. Even a simple traffic light system indicating the level of confidence in a dataset could be useful. The inputs for determining color would be the

Procedure NASA MDP Data Preprocessing Approach

inputs: Data – the original NASA MDP data sets.

Flag – the indicator of whether or not the identical/inconsistent cases are removed, can be TRUE or FALSE.

output: DS' or DS'' – the preprocessed NASA MDP data sets. The former is with identical and inconsistent cases while the latter not.

//DS – a specific data set in Data.

//M – the number of cases in DS; N – the number of features in DS.

//DS.Value[i][j] – the value of feature j for case i in DS.

```

1 Data' = NULL;
2 Remove features MODULE_ID and ERROR_DENSITY, convert ERROR_COUNT into defective or non-defective flags (1/0);
3 for each DS ∈ Data do
4     for i = 1 to M do                                     //step 1: remove cases with implausible values
5         for j = 1 to N do
6             if DS.Value[i][j] is an implausible value then
7                 DS = DS - DS.Value[i][1...N];
8     for i = 1 to M do                                     //step 2: remove cases with conflict feature values
9         if DS.Value[i][1...N] contains conflict feature values then
10             DS = DS - DS.Value[i][1...N];
11     if Flag then
12         for i = 1 to M - 1 do                             //step 3: remove identical cases
13             for k = i + 1 to M do
14                 if DS.Value[i][1...N] ≡ DS.value[k][1...N] then
15                     DS = DS - DS.Value[k][1...N];
16         for i = 1 to M - 1 do                             //step 4: remove inconsistent cases
17             for k = i + 1 to M do
18                 if DS.Value[i][1...N-1] ≡ DS.Value[k][1...N-1] and DS.Value[i][N] ≠ DS.Value[k][N] then
19                     DS = DS - DS.Value[i][1...N];
20                     DS = DS - DS.Value[k][1...N];
21     for i = 1 to M do                                     //step 5: remove cases with missing values
22         for j = 1 to N do
23             if DS.Value[i][j] is a missing value then
24                 DS = DS - DS.Value[i][1...N];
25     for j = 1 to N do                                     //step 6: remove constant features
26         if DS.Value[1...M][j] is constant then
27             DS = DS - DS.Value[1...M][j];
28     for j = 1 to N - 1 do                                 //step 7: remove identical features
29         for k = j + 1 to N do
30             if DS.Value[1...M][j] ≡ DS.Value[1...M][k] then
31                 DS = DS - DS.Value[1...M][k];
32     Data' = Data' ∪ {DS};
33 If (!Flag) then DS' = Data' else DS'' = Data';

```

Fig. 1. Procedure NADA MDP data processin approach.

extent of 1) documentation and 2) reproducibility of results based upon the dataset.

Third, as is evident from Table 5, authors (including ourselves) have not been in the habit of providing complete information regarding preprocessing of data. Given that many reported differences between machine learners are quite modest, the means by which missing values are handled and whether duplicates are removed or inconsistent

values checked matter a good deal. Keung et al. [11] also comment that “ranked estimator lists are highly unstable in the sense that different methods combining with different preprocessors may yield very different rankings, and that a small change of the dataset usually affects the obtained estimator list considerably.” Thus, the trivial detail may have a far reaching impact upon the final results and conclusions. These problems can be addressed by agreed reporting

protocols that need to be developed and owned by the research community.

To conclude, some of the differences and data quality may seem trivial and sometimes impact only a small proportion of the observations. However, if our research is to have the respect of our fellow scientists, then addressing such problems is not optional.

Last, we should stress that the foregoing discussion is not in anyway a criticism of NASA; rather, it raises some questions concerning how we, as a research community, have made use of such data to learn more about predicting defect-prone software modules.

APPENDIX

The data cleaning tool may be downloaded from <http://nasa-softwaredefectdatasets.wikispaces.com/>.

Implausible values

- LOC_TOTAL = 0.
- value of any attribute is < 0.
- any count is a noninteger.

Referential integrity checks

1. NUMBER_OF_LINES $\geq$ LOC_TOTAL
2. NUMBER_OF_LINES $\geq$ LOC_BLANK
3. NUMBER_OF_LINES $\geq$ LOC_CODE_AND_COMMENT
4. NUMBER_OF_LINES $\geq$ LOC_COMMENTS
5. NUMBER_OF_LINES $\geq$ LOC_EXECUTABLE
6. LOC_TOTAL $\geq$ LOC_EXECUTABLE
7. LOC_TOTAL $\geq$ LOC_CODE_AND_COMMENT
8. NUM_OPERANDS $\geq$ NUM_UNIQUE_OPERANDS
9. NUM_OPERATORS $\geq$ NUM_UNIQUE_OPERATORS
10. HALSTEAD_LENGTH = NUM_OPERATORS + NUM_OPERANDS
11. CYCLOMATIC_COMPLEXITY $\leq$ NUM_OPERATORS + 1
12. CALL_PAIRS $\leq$ NUM_OPERATORS
13. HALSTEAD_VOLUME = (NUM_OPERATORS + NUM_OPERANDS) $\times \log_2$ (NUM_UNIQUE_OPERATORS + NUM_UNIQUE_OPERANDS)
14. HALSTEAD_LEVEL = (2/NUM_UNIQUE_OPERATORS) $\times$ (NUM_UNIQUE_OPERANDS/NUM_OPERANDS)
15. HALSTEAD_DIFFICULTY = (NUM_UNIQUE_OPERATORS/2) $\times$ (NUM_OPERANDS/NUM_UNIQUE_OPERANDS)
16. HALSTEAD_CONTENT = HALSTEAD_VOLUME/HALSTEAD_DIFFICULTY
17. HALSTEAD_EFFORT = HALSTEAD_VOLUME $\times$ HALSTEAD_DIFFICULTY
18. HALSTEAD_PROG_TIME = HALSTEAD_EFFORT/18

ACKNOWLEDGMENTS

The authors would like to thank the NASA MDP organization for making their defect datasets publicly available. They also thank David Bowes and David Gray for useful discussions, the authors of the other *TSE* studies for providing additional information concerning their analysis techniques, and the reviewers for their careful reading and constructive comments. This work was supported in part by the National Natural Science Foundation of China under grant 61070006.

REFERENCES

- [1] S. Armstrong, "Significance Tests Harm Progress in Forecasting," *Int'l J. Forecasting*, vol. 23, no. 2, pp. 321-327, 2007.
- [2] J. Bezdek, J. Keller, R. Krishnapuram, L. Kuncheva, and N. Pal, "Will the Real Iris Data Please Stand Up?" *IEEE Trans. Fuzzy Systems*, vol. 7, no. 3, pp. 368-369, June 1999.
- [3] G. Boetticher, *Improving Credibility of Machine Learner Models in Software Engineering*, pp. 52-72. Idea Group Inc., 2007.
- [4] C. Catal and B. Diri, "A Systematic Review of Software Fault Prediction Studies," *Expert Systems with Applications*, vol. 36, no. 4, pp. 7346-7354, 2009.
- [5] W. Fan, F. Geerts, and X. Jia, "A Revival of Integrity Constraints for Data Cleaning," *Proc. VLDB Endowment*, vol. 1, no. 2, pp. 1522-1523, Aug. 2008.
- [6] D. Gray, D. Bowes, N. Davey, Y. Sun, and B. Christianson, "The Misuse of the NASA Metrics Data Program Data Sets for Automated Software Defect Prediction," *Proc. 15th Ann. Conf. Evaluation and Assessment in Software Eng.*, 2011.
- [7] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A Systematic Literature Review on Fault Prediction Performance in Software Engineering," *IEEE Trans. Software Eng.*, vol. 38, no. 6, pp. 1276-1304, Nov./Dec. 2012.
- [8] D. Ince, L. Hatton, and J. Graham-Cumming, "The Case for Open Computer Programs," *Nature*, vol. 482, no. 7386, pp. 485-488, 2012.
- [9] Y. Jiang, B. Cukic, and T. Menzies, "Fault Prediction Using Early Lifecycle Data," *Proc. 18th IEEE Int'l Symp. Software Reliability Eng.*, pp. 237-246, 2007.
- [10] K. Kaminsky and G. Boetticher, "Building a Genetically Engineerable Evolvable Program (GEEP) Using Breadth-Based Explicit Knowledge for Predicting Software Defects," *Proc. IEEE Ann. Meeting Fuzzy Information Processing Soc.*, pp. 10-15, 2004.
- [11] J. Keung, E. Kocaguneli, and T. Menzies, "Finding Conclusion Stability for Selecting the Best Effort Predictor in Software Effort Estimation," *Automated Software Eng.*, vol. 20, pp. 543-567, 2013.
- [12] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking Classification Models for Software Defect Prediction: A Proposed Framework and Novel Findings," *IEEE Trans. Software Eng.*, vol. 34, no. 4, pp. 485-496, July/Aug. 2008.
- [13] G. Liebchen, "Data Cleaning Techniques for Software Engineering Data Sets," doctoral thesis, Brunel Univ., 2011.
- [14] G. Liebchen and M. Shepperd, "Data Sets and Data Quality in Software Engineering," *Proc. Int'l Workshop Predictor Models in Software Eng.*, 2008.
- [15] Y. Liu, T. Khoshgoftaar, and N. Seliya, "Evolutionary Optimization of Software Quality Modeling with Multiple Repositories," *IEEE Trans. Software Eng.*, vol. 36, no. 6, pp. 852-864, Nov./Dec. 2010.
- [16] T. Menzies, J. Greenwald, and A. Frank, "Data Mining Static Code Attributes to Learn Defect Predictors," *IEEE Trans. Software Eng.*, vol. 33, no. 1, pp. 2-13, Jan. 2007.
- [17] T. Menzies and M. Shepperd, "Editorial: Special Issue on Repeatable Results in Software Engineering Prediction," *Empirical Software Eng.*, vol. 17, nos. 1/2, pp. 1-17, 2012.
- [18] M. Shepperd, "Data Quality: Cinderella at the Software Metrics Ball?" *Proc. Second Int'l Workshop Emerging Trends in Software Metrics*, pp. 1-4, 2011.
- [19] Q. Song, Z. Jia, M. Shepperd, S. Ying, and J. Liu, "A General Software Defect-Prone Prediction Framework," *IEEE Trans. Software Eng.*, vol. 37, no. 3, pp. 356-370, May/June 2011.
- [20] V. Stodden, "The Scientific Method in Practice: Reproducibility in the Computational Sciences," MIT Sloan School Working Paper 4773-10, 2010. <http://ssrn.com/abstract=1550193>, Aug. 2012.

- [21] H. Zhang and X. Zhang, "Comments on "Data Mining Static Code Attributes to Learn Defect Predictors"," *IEEE Trans. Software Eng.*, vol. 33, no. 9, pp. 635-636, Sept. 2007.



Martin Shepperd received the PhD degree in computer science from the Open University in 1991 for his work in measurement theory and its application to empirical software engineering. He is a professor of software technology at Brunel University, London, United Kingdom. He has published more than 150 refereed papers and three books in the areas of software engineering and machine learning. He was editor-in-chief of the journal *Information & Software Technology* (1992-2007) and was an associate editor of the *IEEE Transactions on Software Engineering* (2000-2004). He is currently an associate editor of the journal *Empirical Software Engineering*. He was program chair for Metrics '01 and '04 and ESEM '11.



Qinbao Song received the PhD degree in computer science from Xian Jiaotong University, Xian, China, in 2001. He is currently a professor of software technology in the Department of Computer Science and Technology, Xian Jiaotong University, where he is also the deputy director in the Department of Computer Science and Technology. He is also with the State Key Laboratory of Software Engineering, Wuhan University, Wuhan, China. He has authored or coauthored more than 100 refereed papers in the areas of machine learning and software engineering. He is a board member of the *Open Software Engineering Journal*. His current research interests include data mining/machine learning, empirical software engineering, and trustworthy software.



Zhongbin Sun received the BS degree in computer science from Xi'an Jiaotong University, Xi'an, China, in 2010. He is currently working toward the doctoral degree in the Department of Computer Science and Technology, Xi'an Jiaotong University. His research focuses on software engineering and data mining.



Carolyn Mair received the PhD degree for her investigations into spatiotemporal aspects of object feature binding. She is a chartered psychologist and chartered scientist. Her research interests lie in the application of theories from cognitive psychology to improve performance and enhance well-being. She is a subject director for psychology at the London College of Fashion, University of the Arts London, and a principal investigator on an EPSRC-funded project investigating decision-making processes in software personnel. She is also investigating consumer behavior and the effect of appearance on self-esteem in older women.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.